

NeoReception

Enterprise Technical Architecture & Solutions Brief

Lead System Architect Presentation

Version 1.0

Date: June 2026

Classification: Client-Facing / Solutions Architecture

Table of Contents

- NeoReception Enterprise Technical Architecture & Solutions Brief 1**
 - Executive Summary 1
 - 1. Architecture & System Design 1
 - 1.1 High-Level Deployment Architecture 1
 - 1.2 Telephony Ingress & Call Control 2
 - 1.3 System Architecture Diagram 2
 - 1.4 End-to-End Call Sequence Diagram 4
 - 1.5 User Journey & Ingestion Interaction Flow 6
 - 1.6 DevOps & Deployment Pipelines 8
 - 2. Technology Stack & Microservices Breakdown 8
 - 2.1 Technology Stack Matrix 8
 - 2.2 Microservices Breakdown 9
 - 3. AI Integration & Logic 11
 - 3.1 AI Routing & Load-Balancing Pool 11
 - 3.2 Model Context Protocol (MCP) Design 13
 - 4. Enterprise-Grade Standards & Compliance 14
 - 4.1 Enterprise Security Architecture 14
 - 4.2 Data Privacy & Compliance (GDPR/HIPAA Ready) 14
 - 4.3 High Availability & Disaster Recovery 15

NeoReception Enterprise Technical Architecture & Solutions Brief

Lead System Architect Presentation

Version: 1.0 · **Date:** June 2026 · **Classification:** Client-Facing / Solutions Architecture

Platform URL: neoreception.com

Executive Summary

NeoReception is a high-performance, enterprise-grade Software-as-a-Service (SaaS) platform designed to deploy autonomous, AI-powered conversational voice agents for businesses. Built on a resilient, multi-cloud foundation and engineered in Rust, NeoReception connects public telephony carriers to state-of-the-art Realtime Generative AI models.

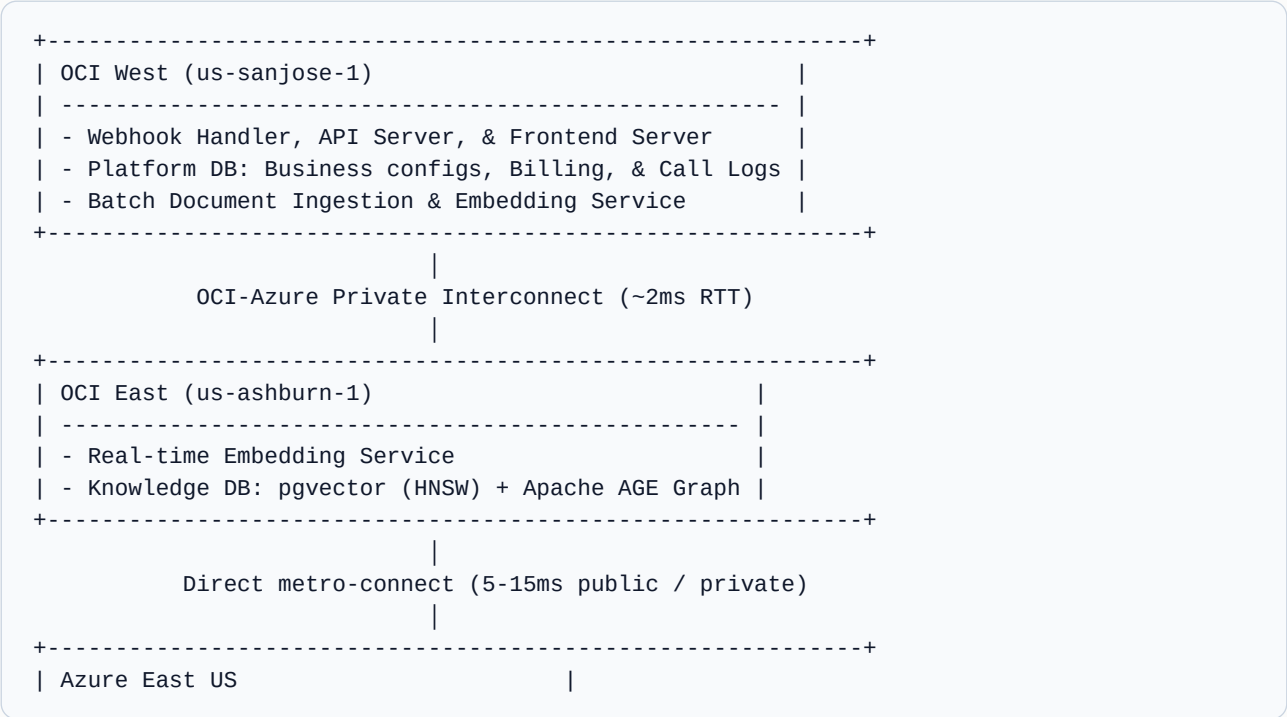
By leveraging carrier-grade **Session Initiation Protocol (SIP)** media routing, **Model Context Protocol (MCP)** knowledge synthesis, and a **dual-instance Oracle Cloud Infrastructure (OCI)** setup, the platform achieves sub-second conversation latency, guaranteed zero-data-loss event handling, and strict enterprise compliance.

This document details the system design, microservices structure, AI reasoning engine, and security compliance protocols that govern the NeoReception production environment.

1. Architecture & System Design

1.1 High-Level Deployment Architecture

NeoReception’s production environment is deployed in a multi-region, dual-instance topology on **Oracle Cloud Infrastructure (OCI)**, co-located near **Azure OpenAI** datacenters to minimize latency.



```
| ----- |  
| - Azure Cognitive Services (TTS/LLM) |  
+-----+
```

1. **OCI West (us-sanjose-1)**: Hosts the primary transactional plane. It contains the [Pingora Gateway], the [Webhook Handler], the [API Server], and the primary **Platform Database** (PostgreSQL 16).
2. **OCI East (us-ashburn-1)**: Hosts the low-latency query plane. It contains the [MCP Knowledge Server], the real-time query **Embedding Service**, and the **Knowledge Database** (PostgreSQL with `pgvector` and `Apache AGE` extensions).
3. **Cross-Cloud Private Interconnect**: OCI East is connected to **Azure East US** via OCI FastConnect and Azure ExpressRoute with FastPath enabled. This private network bridge delivers a stable **~2ms round-trip time (RTT)**, bypassing the public internet and eliminating jitter.

1.2 Telephony Ingress & Call Control

Unlike legacy solutions that route real-time audio through high-overhead WebSocket middleware, NeoReception utilizes **SIP-native media routing** to stream audio directly between the telephony provider (Telnyx/Twilio) and the AI engine.

- **SIP Transfer Protocol**: When an incoming call hits a provisioned DID, the Webhook Handler validates the caller and issues a SIP `transfer` command to the carrier.
- **Direct Media Flow**: The carrier's media gateway streams audio directly to the Azure OpenAI GPT Realtime SIP endpoint via **SRTP**. The OCI backend is completely removed from the audio path during the conversation, reducing server load to zero and saving **50–100ms of transcoding latency**.
- **Apology & Graceful Rejection**: If all AI pools are saturated, the system responds with TwiML/TeXML instructions to play a cached error message and hang up, avoiding dead-air.

1.3 System Architecture Diagram

The following diagram illustrates the routing of call media, webhooks, RAG tools, and database calls across OCI, AWS S3/CloudFront, and Azure OpenAI pools:

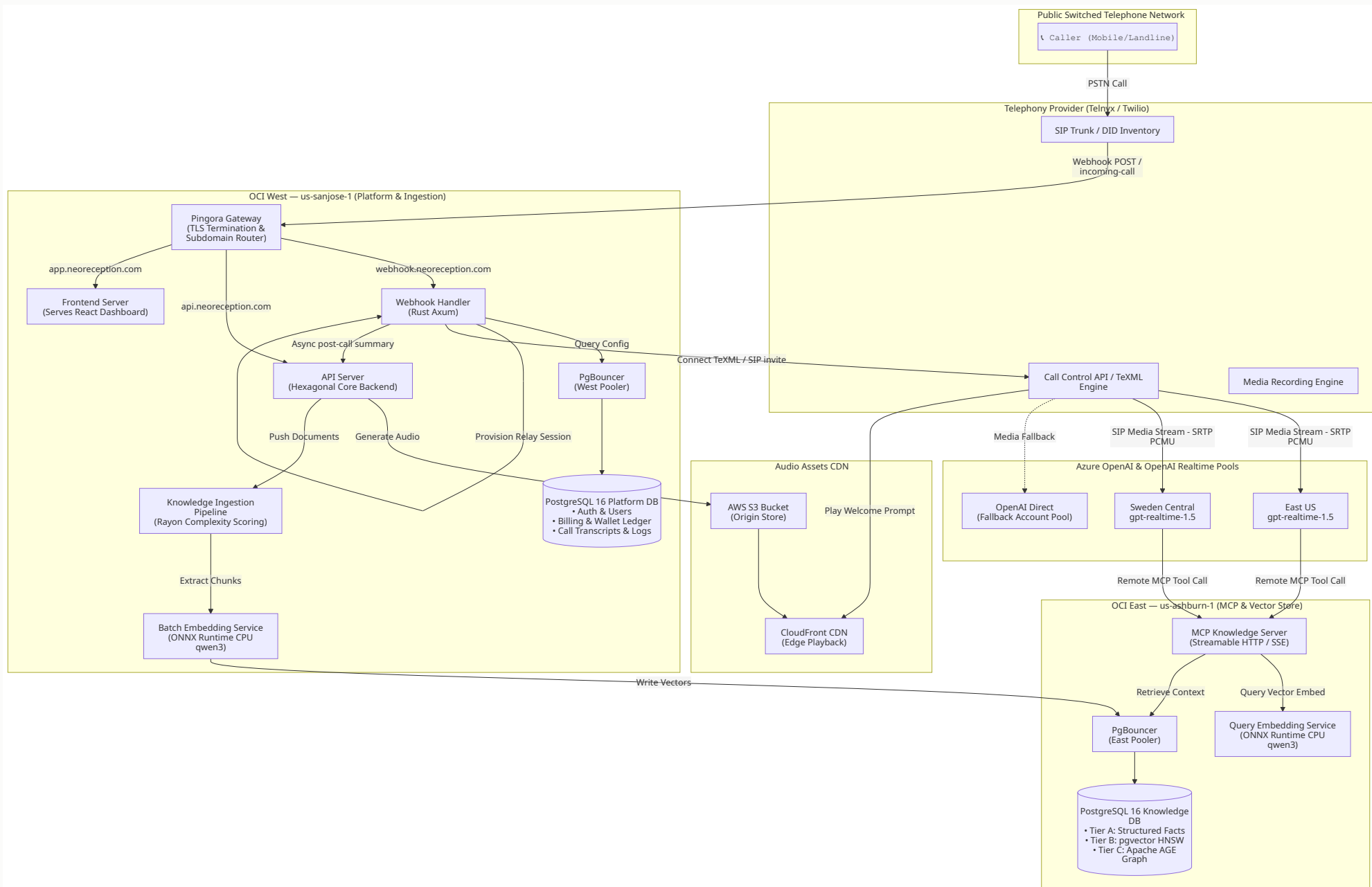


Figure 1: System Architecture Diagram

1.4 End-to-End Call Sequence Diagram

The interaction sequences during call startup, live tool querying, and asynchronous teardown are mapped below:

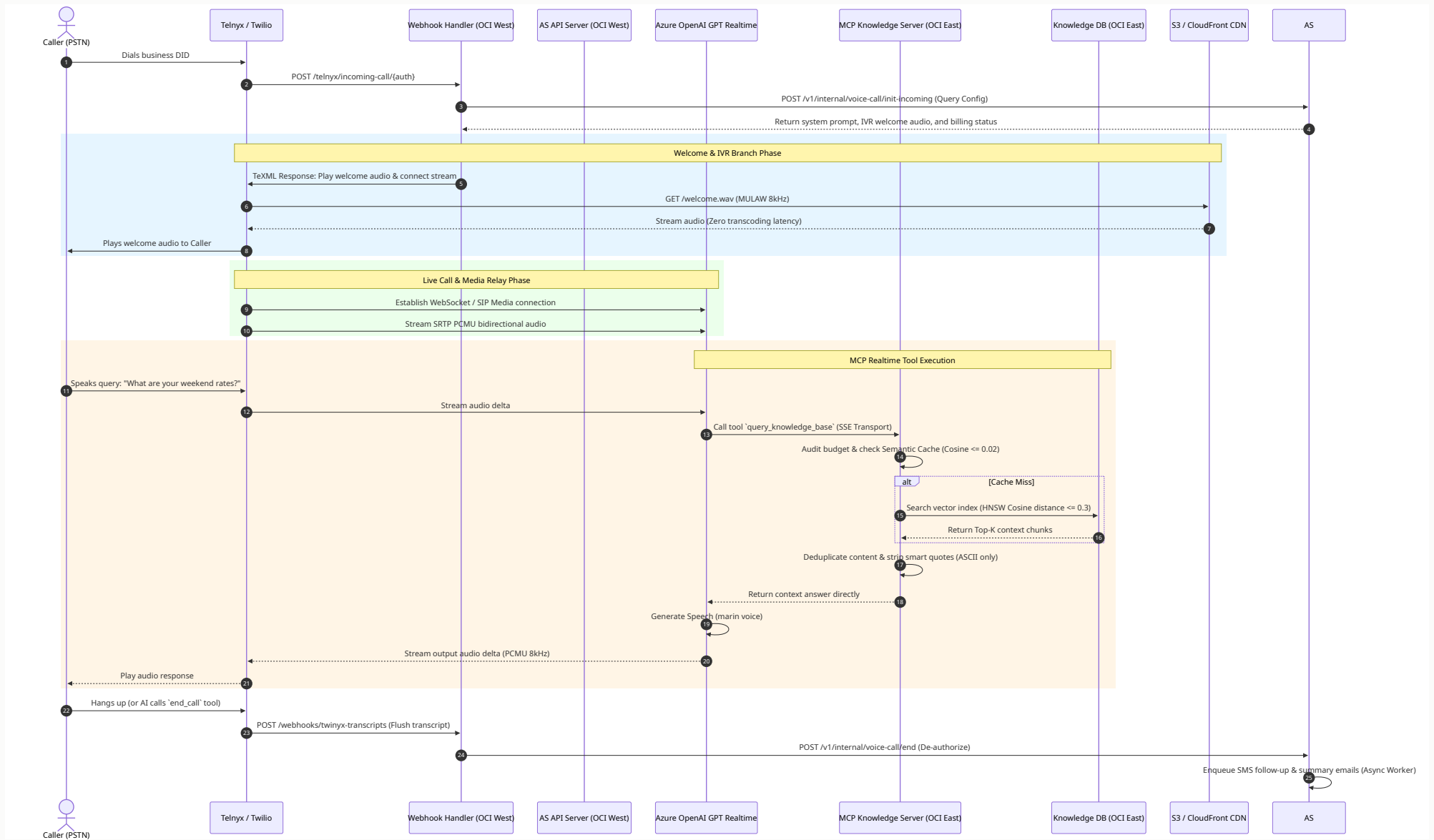


Figure 2: End-to-End Call Sequence Diagram

1.5 User Journey & Ingestion Interaction Flow

This diagram details the path business owners take to provision phone numbers, upload enterprise documentation, and review logs:

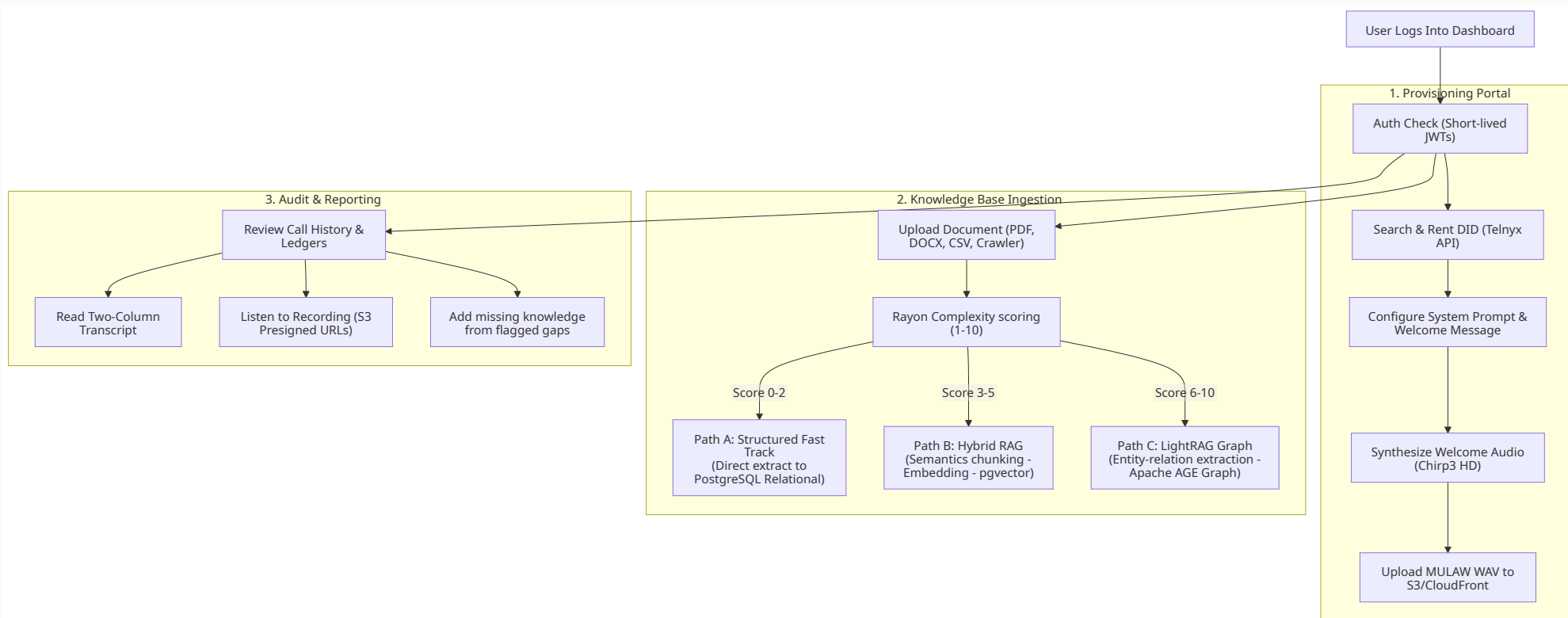
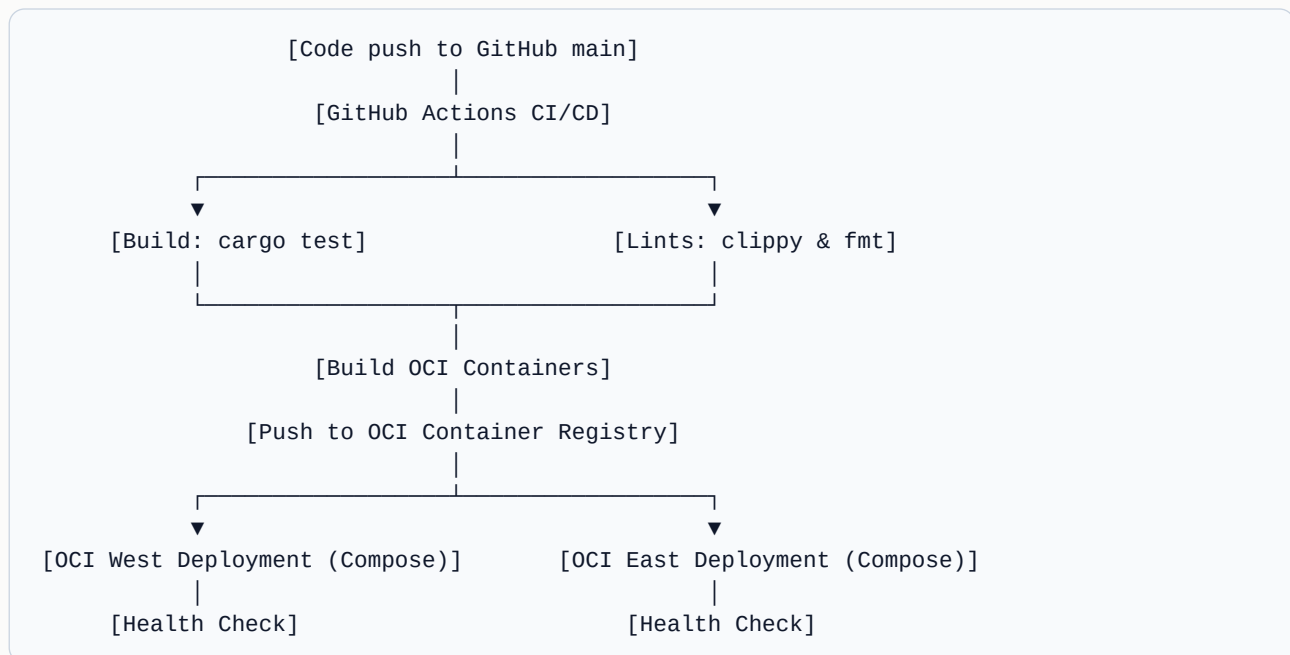


Figure 3: User Journey & Ingestion Interaction Flow

1.6 DevOps & Deployment Pipelines

All services in the NeoReception workspace are packaged as container images and managed with **Podman Compose** to support a “Lean Launch” strategy, facilitating a seamless transition to a Kubernetes orchestration layer in later phases.



- **Containerization:** Microservices are designed container-first. Container specifications, ports, and boundaries remain identical under both Podman Compose and Kubernetes deployments.
- **Continuous Integration:** The GitHub Actions runner executes `cargo test`, `cargo clippy`, and formatting checks before building production images.
- **Zero-Downtime Rolling Deployments:** During deployment, new containers start and pass health-checks before old ones are terminated. The reverse-proxy drains connections gracefully to prevent call drops.

2. Technology Stack & Microservices Breakdown

NeoReception features a microservices architecture engineered to optimize execution speeds, resource usage, and process boundaries.

2.1 Technology Stack Matrix

Architectural Layer	Implemented Technology	Engineering Rationale
Reverse Proxy & Ingress	[Pingora (Cloudflare)]	Rust-native, low-footprint, subdomain routing, and high concurrency without connection bottlenecks.
Backend Framework	Rust + Axum + Tokio	Compilation safety, thread-safe async I/O loops, and compile-time SQL query checking via SQLx.
Real-time Media Relays	[twinyx-azure-relay], [twinyx-gcp-relay]	Low-level WebSocket management, bit-manipulation for codecs, and zero-allocation frame buffering.
Relational Database	PostgreSQL 18+	High transactional integrity, support for complex schemas, and connection multiplexing via Pg-Bouncer.
Vector Indexing	pgvector (HNSW Index)	Keeps semantic embeddings inside the relational database, avoiding multi-database synchronization bugs.
Knowledge Graph	Apache AGE (PostgreSQL)	Allows running Cypher queries over graph nodes stored natively within PostgreSQL.
Inference Engines	Azure OpenAI (gpt-real-time-1.5), Google Gemini (Live API)	Native support for real-time, low-latency audio inputs and outputs.
Pre-generated Speech	Google Cloud TTS (Chirp3 HD)	Synthesizes custom welcome messages directly in MULAW 8kHz format, eliminating transcoding during calls.
Self-Hosted Embeddings	ONNX Runtime + Qwen-3 (INT8)	Bypasses third-party embedding API costs and network latency (~5ms execution on CPU).
Secrets & Keys	OCI Vault	Hardware Security Module (HSM) protection for API keys and database credentials.

2.2 Microservices Breakdown

2.2.1 Pingora Gateway Proxy (`pingora-gateway`)

- **Language/Framework:** Rust + Pingora (Cloudflare's HTTP proxy library)
- **Role:** Serves as the primary ingress point. Terminates TLS using Let's Encrypt certificates and performs subdomain routing (`app.*` , `api.*` , `webhook.*`).
- **Engineering Rationale:** Replacing Nginx with Pingora aligns our entire infrastructure under the Rust toolchain, reducing memory overhead and allowing custom proxy filters to be written in Rust.

2.2.2 OCI Webhook Handler (`twinyx-oci-webhook`)

- **Language/Framework:** Rust + Axum + SQLx

- **Role:** Receives and processes carrier webhooks (`/twilio/incoming-call/{auth}` and `/telnyx/incoming-call/{auth}`).
- **Features:** Performs HMAC signature validation, queries agent configurations, validates STIR/SHAKEN caller authenticity, checks spam scores, checks billing status, and generates TeXML/TwiML response payloads.
- **Engineering Rationale:** Kept completely stateless to allow horizontal scaling behind the gateway.

2.2.3 Voice Call Service WebSocket Controller (`voice-call-service`)

- **Language/Framework:** Rust + Axum (Tokio)
- **Role:** Manages active sessions for browser-based dashboard calls.
- **Features:** Upgrades client connections to WebSockets and proxies media packets directly to the internal relays.
- **Implementation:** Utilizes a Rust **RAII Cleanup Guard** (`CallCleanupGuard`) that intercepts task drop events (e.g., client disconnection or crash) to ensure the backend API is notified to terminate billing immediately.
- **Engineering Rationale:** Implements a strict per-minute billing tick worker that terminates connections if the business account balance falls below zero.

2.2.4 Real-time Media Relays (`twinyx-azure-relay` & `twinyx-gcp-relay`)

- **Language/Framework:** Rust + Tokio WebSocket + Tungstenite
- **Role:** Bridges G.711 μ -law 8kHz audio streams from telephony channels with the Generative AI Live APIs.
- **Audio pipeline:** Resamples incoming 8kHz μ -law streams to 24kHz PCM16 (required by Azure OpenAI) and downsamples outgoing 24kHz streams back to 8kHz μ -law (required by telephony networks).
- **Engineering Rationale:** Written in low-level Rust to bypass intermediate audio-processing frameworks (like FFmpeg), ensuring sub-millisecond audio frame processing.

2.2.5 Knowledge Retrieval MCP Server (`knowledge-retrieval-mcp`)

- **Language/Framework:** Rust + `rust-mcp-sdk` + Hyper
- **Role:** Exposes the business knowledge base to the AI agents via the Model Context Protocol (MCP).
- **Exposed Tool:** `query_knowledge_base`
- **Features:** Pre-flight atomic token budget reservation, exact text caching, semantic vector caching, direct context routing, and TTS-safe character sanitization.
- **Engineering Rationale:** Implements dual-transport endpoints: `/sse` for OpenAI's legacy remote tools and `/mcp` (Streamable HTTP) for standard MCP clients (like Cursor or Claude Desktop).

2.2.6 Knowledge Ingestion Pipeline (`knowledge-ingestion`)

- **Language/Framework:** Rust + Rayon + `pdf_oxide` + `docx-rust`
- **Role:** Extracts text from business uploads (PDFs, Word documents, URLs, scanned images) and builds vector databases and knowledge graphs.
- **Features:** Parallelized complexity scoring using CPU thread pools (Rayon) to determine the extraction path.

- **Extraction Paths:**

- **Path A (Score <= 2):** Structured data extracted via GPT-4o mini and stored directly in PostgreSQL relational tables.
- **Path B (Score 3-5):** Text chunked (500 tokens, 50 overlap) and indexed in pgvector.
- **Path C (Score >= 6):** Entity-relationship map extracted via LightRAG and stored in the Apache AGE graph database.

2.2.7 Embedding Service (`embedding-service`)

- **Language/Framework:** Rust + ONNX Runtime (CPU)
- **Role:** Computes vector embeddings for document ingestion and incoming queries.
- **Model:** `qwen3-embedding-0.6` quantized to 8-bit integer (INT8) format.
- **Engineering Rationale:** Bypasses the network latency and cost of cloud APIs. Achieves ~5-15ms local inference speeds by utilizing CPU SIMD (NEON) instructions.

2.2.8 GCP TTS Service (`twinyx-gcp-tts`)

- **Language/Framework:** Rust + Google Cloud Speech SDK
- **Role:** Generates welcome audio prompts in G.711 μ -law 8kHz format.
- **Engineering Rationale:** Pre-synthesized welcome messages are cached on the CDN to eliminate text-to-speech generation latency during call start.

2.2.9 API Core Server (`api-server`)

- **Language/Framework:** Rust + Axum (Hexagonal Architecture)
- **Role:** Exposes REST APIs for user authentication, dashboard management, billing transactions, and analytics.

3. AI Integration & Logic

3.1 AI Routing & Load-Balancing Pool

Because the Azure OpenAI `gpt-realtime-1.5` API enforces strict concurrent connection limits per region, NeoReception implements a **Weighted round-robin load-balancing registry** to pool resources:

```
Score(endpoint) = (1 - current_utilization) × static_weight × health_factor
```

Where:

```
current_utilization = active_rpm / rpm_limit
```

```
health_factor       = 1.0 (Healthy), 0.5 (Degraded), 0.0 (Unhealthy)
```

The system monitors regional load and routes calls dynamically: 1. **Geo-Affinity Routing:** Calls are routed to the nearest regional datacenter (e.g., North American calls default to East US, European calls default to Sweden Central). 2. **Subscription Spillover:** If the primary region's capacity reaches 75% (soft limit), the router shifts traffic to secondary subscriptions. 3. **Cross-Cloud Fallback:** If an entire cloud provider fails, the router falls back to OpenAI Direct or Vertex AI, ensuring high call availability.

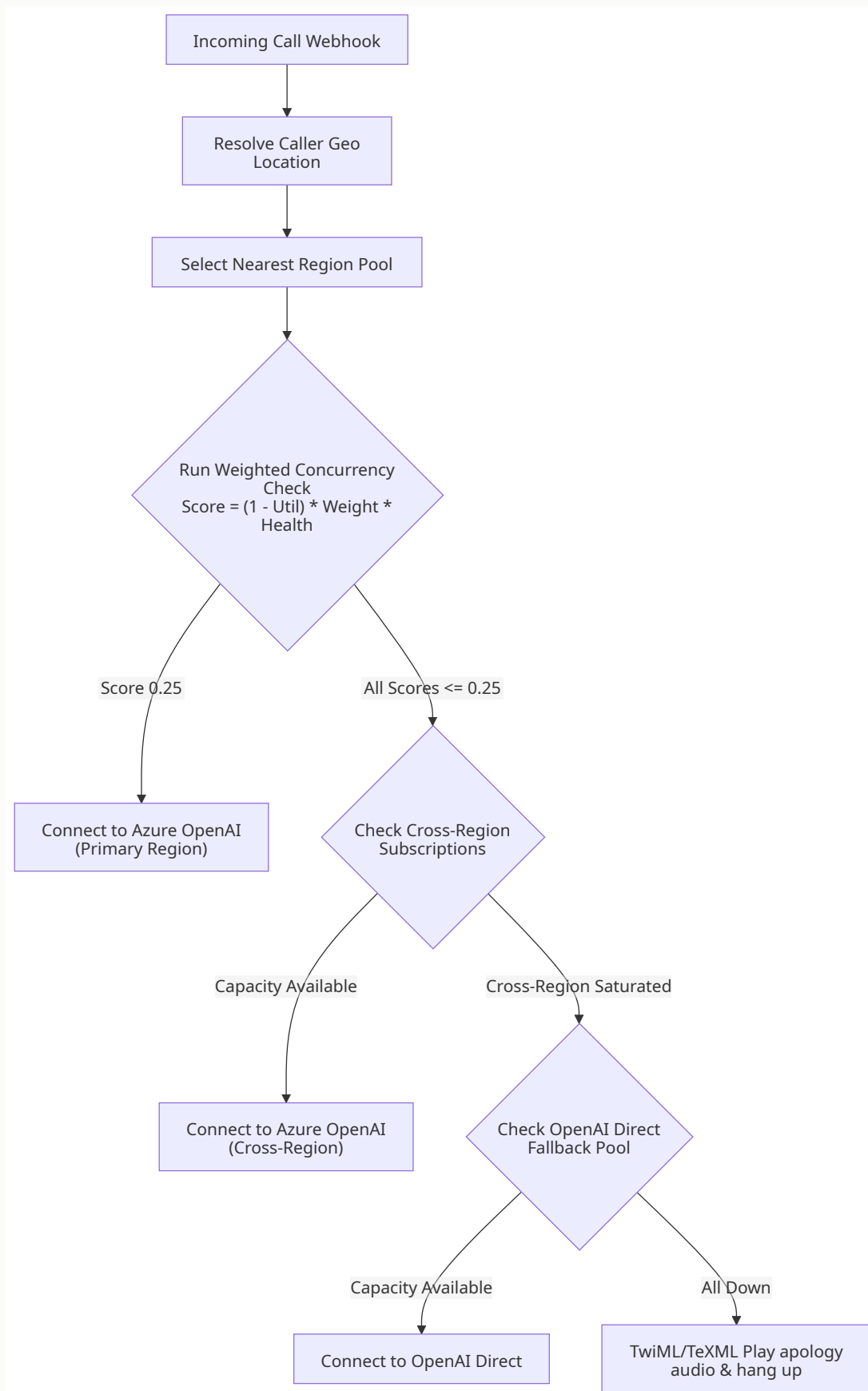
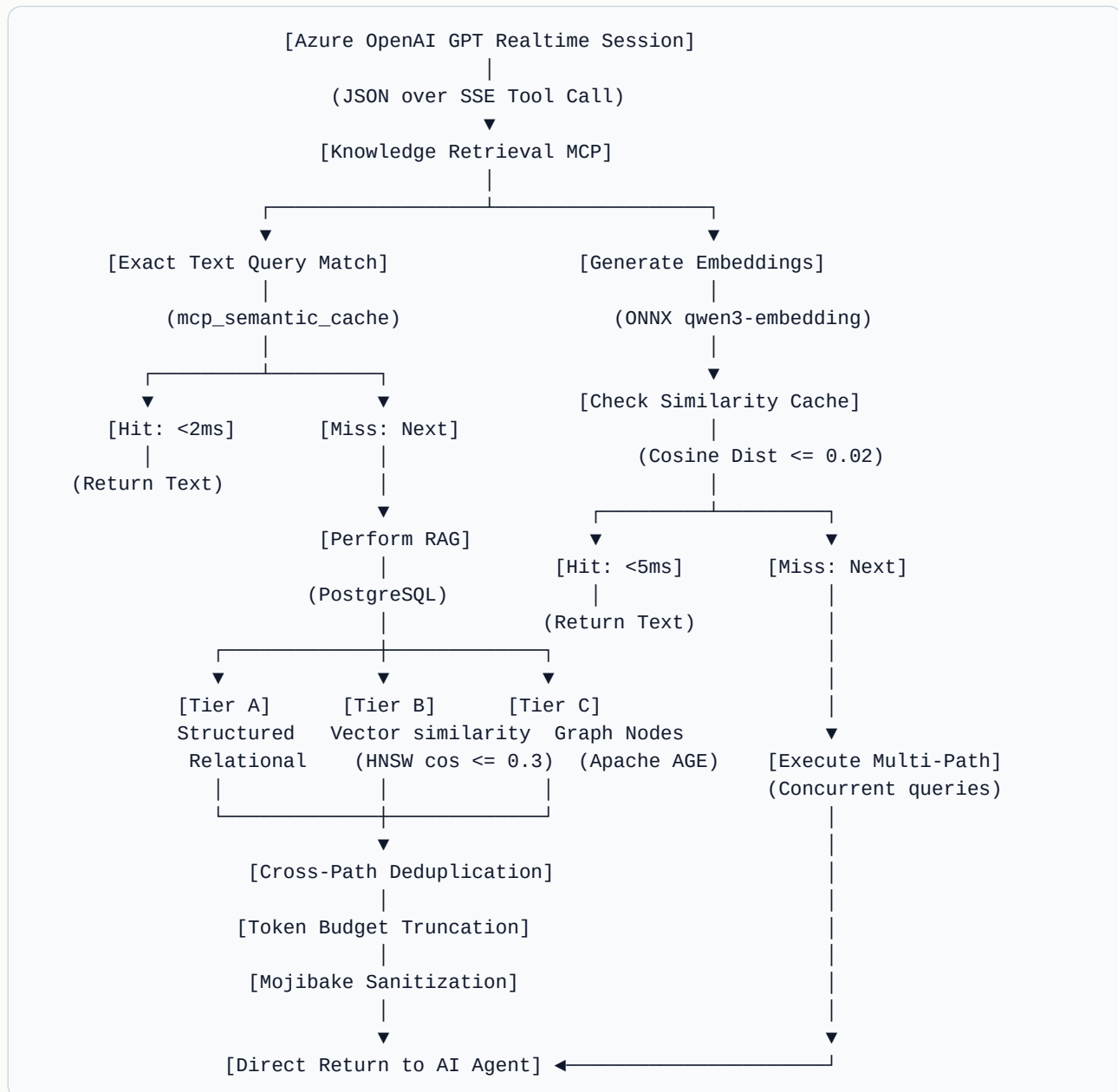


Figure 4: AI Routing & Load-Balancing Pool

3.2 Model Context Protocol (MCP) Design



The MCP Knowledge Server exposes the `query_knowledge_base` tool to provide agents with real-time access to business data:

3.2.1 Pre-Flight Budget Reservation

To prevent concurrent queries from exceeding a tenant's daily token allocation, the MCP server runs an atomic check against usage tables before executing the RAG pipeline. If the limit is exceeded, it returns a friendly notice to the agent, which handles the response gracefully.

3.2.2 Dual-Tier Caching System

To optimize response times and lower API costs, queries are routed through a two-tiered caching layer:

- Tier 1 (Exact Match Cache):** Checks the `mcp_semantic_cache` table for exact string matches. Hits bypass the embedding and retrieval steps, resolving in **<2ms** at **\$0 cost**.
- Tier 2 (Semantic Cache):** Computes the query embedding and runs a cosine distance search (`query_embedding <=> query_embedding <= 0.02`). Hits resolve in **<5ms**.

3.2.3 Direct Return & Deduplication

If the query misses the cache, the server queries the database across all three knowledge tiers (relational facts, vector chunks, and graph relations) concurrently using `tokio::join!`. - **Deduplication:** Content is deduplicated using SipHash fingerprints to eliminate redundant facts. - **Direct Return:** Retrieved context is returned directly to the agent, bypassing intermediate LLM reformulation calls. This reduces tool-call latency from **~1500ms to <150ms**, saves API costs, and eliminates downstream hallucination risks.

3.2.4 Telephony Audio Codec & TTS Sanitization

The system sanitizes RAG responses to prevent speech synthesis glitches: - **ASCII Sanitization:** Strips non-ASCII symbols, formatting characters, and encoding artifacts (like smart quotes) from the text context. - **Telnyx-Native Formatting:** Welcome prompts are pre-compiled into WAV containers using 8kHz μ -law (PCMU) encoding, allowing the telephony carrier to play them directly without transcoding.

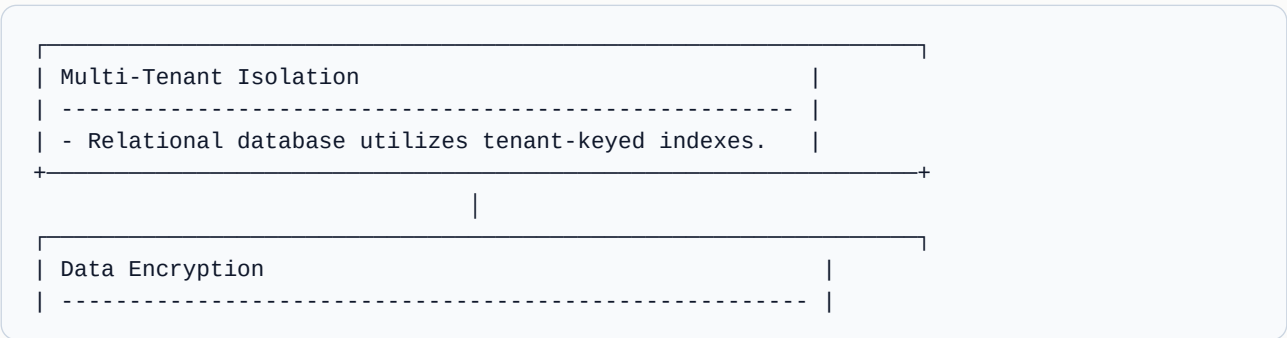
4. Enterprise-Grade Standards & Compliance

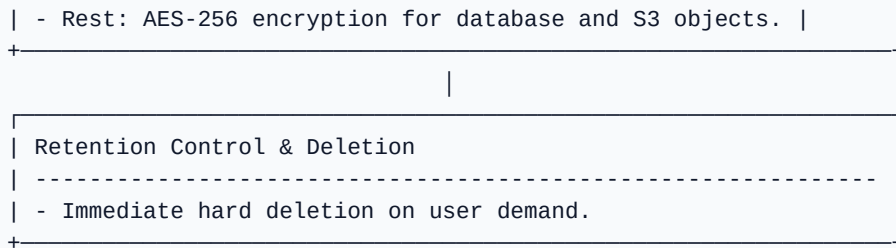
4.1 Enterprise Security Architecture

NeoReception implements secure, production-ready access control across all channels:

- **Telephony Webhook Security:** Incoming webhooks from carrier systems are verified using SHA-256 HMAC signatures (`X-Twilio-Signature` / `X-Telnyx-Signature`). Unauthorized requests are rejected immediately.
- **Spam & Fraud Defense:**
 - **CAPTCHA:** The AI agent prompts unknown numbers to verify their identity on their first call (e.g., “Press 3 to continue”). Verified callers are added to a bypass list.
 - **Nomorobo & STIR/SHAKEN:** Integrates carrier validation tools. Calls with failing STIR/SHAKEN signatures or high spam scores are automatically blocked.
- **Identity & Access Management (IAM):**
 - Inside OCI, services communicate using GCP Workload Identity Federation or Azure AD Managed Identities, eliminating static API keys.
 - Dashboard API requests require short-lived, cryptographically signed JSON Web Tokens (JWTs).
- **Secrets Encryption:** Secrets, database credentials, and API keys are stored in an HSM-backed OCI Vault.

4.2 Data Privacy & Compliance (GDPR/HIPAA Ready)





- **GDPR Compliance:** NeoReception supports the “Right to be Forgotten.” Users can export or delete their entire call log history, contact information, and voice recordings.
- **Data Protection:**
 - **In Transit:** All database, API, and WebSocket traffic is encrypted using TLS 1.3. Real-time media streams utilize secure SRTP.
 - **At Rest:** PostgreSQL databases and S3 Welcome audio files are encrypted at rest using AES-256.
- **Payment Compliance:** Payment processing is handled via Stripe. Stored payment profiles are tokenized on PCI-compliant servers, ensuring no raw credit card data enters the OCI infrastructure.

4.3 High Availability & Disaster Recovery

NeoReception targets carrier-grade availability (99.99%) through redundancy and fail-safe designs:

- **Stateless Application Servers:** Webhook and API nodes run without local state. If a node fails, replacement instances spin up in under 60 seconds with zero call interruption.
- **PostgreSQL Database Redundancy:**
 - Employs PgBouncer to manage connection spikes during call volume peaks.
 - The Platform and Knowledge databases run continuous WAL archiving to OCI Object Storage, supporting point-in-time recovery (PITR) with an **RPO of < 1 minute**.
- **System Recovery Targets:**
 - **Call Media Routing (RTO < 30s):** Managed by carrier-side failover to alternative regional gateways.
 - **Durable Database Store (RTO < 5m):** Achieved via warm database standby clusters.